Adaptive Budget Scheduling for Constrained RL Deployment via Energy Composition

Julian Quick
DTU Wind Energy
Technical University of Denmark
juqu@dtu.dk & $MarcusBinderNilsen$
DTU Wind Energy
Technical University of Denmark

April 12, 2026

Abstract

Deploying reinforcement learning policies in the real world often requires cumulative operational constraints unknown at training time—maintenance budgets, emission limits, duty-cycle restrictions. Standard constrained MDP approaches require full retraining for each constraint specification. We derive a closed-form, time-varying penalty weight from a budget-balanced sequential allocation problem under Boltzmann policy response: the optimal multiplicative weight is $w^*(u) = 1/u$, where u is the ratio of remaining budget fraction to remaining time fraction. We propose a practical schedule $w(u) = \exp(\eta(1/u - 1))$ that matches the optimum to first order, adds a tunable risk-aversion parameter η, and composes post-hoc with pre-trained energy-based or diffusion policies via per-agent energy addition—requiring zero retraining. Across two domains—wind farm yaw control and MuJoCo locomotion with velocity budgets—the method satisfies cumulative budgets while retaining 77–96% of unconstrained reward, outperforming constant-penalty baselines by 7–20 percentage points in budget utilization. A single pre-trained policy supports arbitrary budget levels, per-agent heterogeneous constraints, and instant adaptation—all without retraining.

1 Introduction

Consider a wind farm operator who trains a reinforcement learning (RL) controller to maximize power production. Six months after deployment, a bearing inspection reveals that one turbine needs restricted yaw travel: it may operate at negative yaw angles for at most 15 hours per year to limit structural fatigue. Retraining the controller is expensive and disruptive. The operator needs a way to impose this new cumulative constraint on the *existing* policy, instantly and without retraining.

This scenario is ubiquitous in deployed RL systems. Cumulative operational constraints arise whenever an agent must limit the total time spent in a high-reward but costly regime: wind turbines with load budgets [van Dijk et al., 2025], robots with joint-wear maintenance cycles, grid batteries with cycle-life limits, and advertisers with daily spending caps [Balseiro et al., 2024]. Unlike instantaneous constraints (avoid obstacles, stay upright), these are *lifetime budgets*: the agent must decide *when* to spend its limited capacity for risky actions, allocating strategically across time.

This budget allocation problem is structurally analogous to optimal trade execution in quantitative finance [Almgren and Chriss, 2001]: a trader must liquidate a position over a time horizon, deciding when to trade aggressively (capturing favorable prices) versus passively (conserving capacity). The key insight from that literature—that the optimal execution rate depends on remaining inventory relative to remaining time—transfers directly to RL budget management.

Existing approaches to constrained RL fall into two categories, neither of which solves this problem: (1) **Constrained MDP methods** [Altman, 1999, Stooke et al., 2020, Sootla et al., 2022, Chen et al., 2025] learn a Lagrangian dual variable during training, requiring full retraining for each new constraint specification. (2) **Post-hoc constraint composition** [Urain et al., 2023, Zhang et al., 2025, Xiao et al., 2025] adds constraints at inference time via energy composition or classifier guidance, but handles only *instantaneous* constraints (barrier functions, geometric limits)—not cumulative budgets that span an episode.

We fill this gap. Our contributions are:

1. A **closed-form time-varying penalty weight** derived from budget-balanced sequential allocation under Boltzmann policy response (Section 3). The optimal weight $w^*(u) = 1/u$ depends only on the urgency ratio $u =$ (budget remaining/budget total)/(time remaining/horizon).
2. A **practical schedule** $w(u) = \exp(\eta(1/u - 1))$ that approximates the optimum to first order, recovers uniform allocation at $\eta = 0$, and provides tunable risk aversion via a single scalar parameter (Section 4).
3. **Post-hoc composition** with pre-trained energy-based transformer (EBT) actors via per-agent energy addition, enabling zero-retraining constraint enforcement with heterogeneous per-agent budgets (Section 4.1).
4. **Empirical validation** across two domains—wind farm yaw control and MuJoCo locomotion—showing budget satisfaction, reward retention, and superiority over constant-penalty baselines (Section 6).

2 Background

2.1 Constrained MDPs and Lagrangian Relaxation

A Constrained MDP (CMDP) [Altman, 1999] augments the standard MDP objective with cost constraints:

$$\max_{\pi} \ \mathbb{E}_\pi\left[\sum_{t=0}^{T-1} r(s_t, a_t)\right] \quad \text{s.t.} \quad \mathbb{E}_\pi\left[\sum_{t=0}^{T-1} c^i(s_t, a_t)\right] \le B^i \quad \forall\, i \tag{1}$$

where c^i is a per-agent cost indicator and B^i is the lifetime budget for agent i. The standard approach introduces a Lagrangian dual variable μ:

$$\mathcal{L}(\pi, \mu) = \mathbb{E}_\pi\left[\sum_t r(s_t, a_t)\right] - \mu \cdot \left(\mathbb{E}_\pi\left[\sum_t c(s_t, a_t)\right] - B\right) \tag{2}$$

and alternates between policy optimization (primal) and multiplier updates via dual gradient ascent: $\mu_{k+1} = \max(0,\, \mu_k + \alpha(\hat{C}_k - B))$ [Stooke et al., 2020]. This requires retraining the policy whenever B changes.

2.2 Energy-Based RL and Post-Hoc Composition

Energy-Based Transformer (EBT) actors [Gladstone et al., 2025] generate actions by gradient descent on a learned energy landscape. Given per-agent embeddings $\mathbf{e}^i$ from a transformer encoder, an energy head $E^i_\phi(\mathbf{e}^i, a^i)$ assigns scalar energy to candidate actions. Actions are optimized via:

$$a^{(k+1)} = \text{clip}\Big(a^{(k)} - \eta_{\text{lr}} \nabla_a E_{\text{total}}(a^{(k)}),\ -1,\ 1\Big) \tag{3}$$

The key property enabling post-hoc constraints: external energy terms compose *per-agent, before aggregation*:

$$E_{\text{total}}(a) = \frac{1}{N}\sum_{i=1}^{N}\big[E^i_\phi(\mathbf{e}^i, a^i) + \lambda_{\text{gs}} \cdot E^i_{\text{constraint}}(a^i)\big] \tag{4}$$

This enables adding new constraints at inference time without modifying E_ϕ. Prior work composes instantaneous constraints (*e.g.*, yaw limits, obstacle avoidance) [Urain et al., 2023, Zhang et al., 2025]. We extend this to *cumulative budget constraints* via a time-varying energy weight.

3 Optimal Budget Scheduling

We derive the penalty weight from a sequential budget allocation problem.

Setup. An agent must allocate B "risky" actions over T time steps. At each step t, the agent observes conditions and decides whether to spend one unit of budget. Spending yields a time-dependent benefit; the goal is to maximize total benefit subject to the budget constraint.

Budget tracking. Let $C_t^i = \sum_{s=0}^{t-1} c^i(a_s)$ denote the cumulative cost for agent i at time t, where $c^i(a) = \mathbf{1}[a^i \in \mathcal{A}_{\text{risky}}]$ is the cost indicator (*e.g.*, $\mathbf{1}[\text{yaw} < 0]$ for wind farms, $\mathbf{1}[\text{velocity} > v_{\text{thresh}}]$ for locomotion). Define the *urgency ratio*:

$$u_t^i = \frac{\beta_t^i}{\tau_t}, \qquad \beta_t^i = \frac{B^i - C_t^i}{B^i}, \qquad \tau_t = \frac{T-t}{T} \tag{5}$$

where β_t^i is the remaining budget fraction and τ_t is the remaining time fraction. When $u > 1$, the agent has spent less than the uniform (TWAP) rate; when $u < 1$, it has overspent.

Proposition 1 (Optimal penalty weight under Boltzmann response)**.** *Assume the policy responds to penalty weight λ via a Boltzmann model: $P(\text{risky} \mid \lambda) = P_0 \exp(-\alpha\lambda)$. Budget balance over the remaining horizon requires $\tau \cdot P_0 \exp(-\alpha\lambda) = b$, where b is the remaining budget. The multiplicative weight relative to the TWAP baseline is:*

$$w^*(u) = \frac{1}{u} \tag{6}$$

Proof. At the TWAP point ($u = 1$), budget balance gives $\lambda_0 = \frac{1}{\alpha} \ln \frac{TP_0}{B}$. For general budget state: $\lambda = \frac{1}{\alpha} \ln \frac{\tau P_0}{b}$. The multiplicative weight $w = \exp(\alpha(\lambda - \lambda_0)) = \frac{b/B}{\tau/T}^{-1} = 1/u$. □

Practical schedule. Since $1/u \to \infty$ as $u \to 0$ (budget depleted), we propose the approximation:

$$w(u) = \exp\left(\eta \cdot \left(\frac{1}{u} - 1\right)\right) \tag{7}$$

Corollary 1 (First-order approximation)**.** *Taylor expansion around $u = 1$: $w(u) = 1 + \eta(1/u - 1) + O((1/u - 1)^2)$, matching $1/u = 1 + (1/u - 1)$ to first order when $\eta = 1$.*

The parameter η controls risk aversion: $\eta = 0$ recovers the constant-penalty TWAP baseline (no adaptation); $\eta = 1$ matches the Boltzmann optimum to first order; $\eta > 1$ provides a more conservative safety margin.

Hard wall backstop. A steep exponential barrier activates when budget drops below 5%:

$$\lambda_{\text{wall}}^i(t) = \exp\left(k \cdot \text{ReLU}\left(1 - \frac{\beta_t^i}{0.05}\right)\right) \tag{8}$$

The combined weight is $\lambda^i(t) = \min(w(u_t^i) \cdot \lambda_{\text{wall}}^i(t),\ \lambda_{\max})$, clamped at $\lambda_{\max} = 10^6$.

4 Method: Composable Budget Constraints

4.1 Per-Agent Constraint Energy

The constraint energy for agent i at time t is:

$$E_{\text{budget}}^i(a^i, t) = \lambda^i(t) \cdot \big(\exp\big(k \cdot \text{ReLU}(f_{\text{cost}}(a^i))\big) - 1\big) \tag{9}$$

Algorithm 1 Post-Hoc Budget-Constrained Inference

Require: Trained actor E_ϕ, budget B^i per agent, horizon T, risk aversion η, guidance scale λ_{gs}

1: Initialize $C^i \leftarrow 0$ for all agents i
2: **for** $t = 0, \ldots, T-1$ **do**
3: Observe state s_t, compute embeddings $\mathbf{e}_t^i$
4: Compute urgency $u_t^i = \frac{(B^i - C^i)/B^i}{(T-t)/T}$ for each agent
5: Compute $\lambda^i(t) = \min(\exp(\eta(1/u_t^i - 1)) \cdot \lambda_{\text{wall}}^i,\ \lambda_{\max})$
6: Generate action via gradient descent on $E_\phi + \lambda_{\text{gs}} \cdot E_{\text{budget}}$
7: Execute a_t, observe r_t
8: Update $C^i \leftarrow C^i + c^i(a_t)$ {budget tracking}
9: **end for**

where f_{cost} maps actions to a scalar cost excess (*e.g.*, $f_{\text{cost}}(a) = -a - \theta$ for negative yaw, or $|a| - a_{\text{thresh}}$ for velocity). The base penalty is zero when the action is "safe" and grows exponentially for "risky" actions. The time-varying weight $\lambda^i(t)$ modulates the penalty strength based on remaining budget.

4.2 Composed Energy Landscape

During inference, the total energy optimized by the EBT actor is:

$$E_{\text{total}}(a, t) = \frac{1}{N} \sum_{i=1}^{N} \left[E_\phi^i(\mathbf{e}_t^i, a^i) + \lambda_{\text{gs}} \cdot E_{\text{budget}}^i(a^i, t) \right] \tag{10}$$

No retraining is needed: E_ϕ is frozen from unconstrained training, and the budget term is added post-hoc. The guidance scale λ_{gs} controls overall constraint strength.

4.3 Algorithm

The only additions to the standard EBT inference loop are lines 4–5 (urgency and weight computation, a few floating-point operations) and line 8 (one comparison per agent). The computational overhead is negligible.

4.4 Heterogeneous Per-Agent Budgets

The framework naturally supports per-agent budgets B^i by computing u_t^i independently for each agent. This enables scenarios where agents have different operational histories (*e.g.*, a turbine with a recently replaced bearing can tolerate more aggressive yaw).

4.5 Connection to CMDP Lagrangian Duality

The time-varying weight $\lambda^i(t)$ serves a similar functional role to the Lagrangian dual variable μ in (1): it penalizes constraint-violating actions proportionally to how scarce the remaining budget is. The key difference: μ is learned via dual gradient ascent (requiring retraining), while $\lambda^i(t)$ is computed in closed form from the current budget state. This enables instant adaptation to new budget specifications without any optimization.

5 Experimental Setup

We evaluate the budget scheduling mechanism across two domains with different risky-action semantics.

5.1 Domain 1: Wind Farm Yaw Control

Environment. WindGym [Nilsen et al., 2025] with PyWake backend, 3-turbine farm in a row-aligned layout. Wind speed sampled from 10–14 m/s, wind direction from 225–315°, providing natural temporal variation in wake conditions.

Risky action. Negative yaw angles ($a^i < 0$) steer the rotor against the conventional direction, deflecting wakes to increase downstream power but imposing structural loads on the yawed turbine [van Dijk et al., 2025].

Budget. B = total timesteps at negative yaw per turbine per episode.

Agent. Energy-Based Transformer (EBT) actor with Soft Actor-Critic (SAC), trained unconstrained for 100k steps. The budget surrogate composes post-hoc via per-turbine energy addition at inference.

5.2 Domain 2: MuJoCo Locomotion

Environment. HalfCheetah-v5 [Todorov et al., 2012] with a time-varying reward multiplier $m(t) = 1 + A\sin(2\pi t/T_p)$ ($A = 0.5$, $T_p = 200$) that creates temporal structure: forward progress is worth more during high-multiplier phases.

Risky action. Exceeding a velocity threshold (set at the 50th percentile of unconstrained velocity).

Budget. B = total timesteps above the velocity threshold.

Agent. SAC with Gaussian actor, trained unconstrained for 100k steps. The budget mechanism modulates action magnitudes via a penalty scaling factor at inference.

5.3 Baselines

- **Unconstrained**: no budget constraint (upper bound on reward).
- **Hard clip**: risky action banned entirely (lower bound on reward with budget).
- **Constant penalty** ($\eta = 0$): fixed penalty weight, no adaptation to budget state. Equivalent to a static Lagrangian multiplier.
- **AC schedule** ($\eta > 0$): our time-varying weight with risk aversion $\eta \in \{0.5, 1, 2, 5\}$.

5.4 Ablations

1. **Hard guard vs. AC schedule**: Does the schedule alone prevent budget violations, or is the hard guard ($C^i < B^i$) necessary?
2. $1/u$ **vs.** $\exp(\eta(1/u - 1))$: Theoretically optimal schedule vs. practical approximation.
3. **Budget flexibility**: Same policy at budget levels 10%, 15%, 25%, 50%, 75%.
4. **Boltzmann response validation**: Is the spending rate approximately exponential in the penalty weight?

6 Results

We present results across both domains, focusing on budget satisfaction, reward retention, and the physical understanding of when the agent chooses to incur damage. Figure 1 illustrates how the penalty weight $\lambda(t)$ adapts during one HalfCheetah episode, and Figure 2 shows when the agent decides to spend its budget.

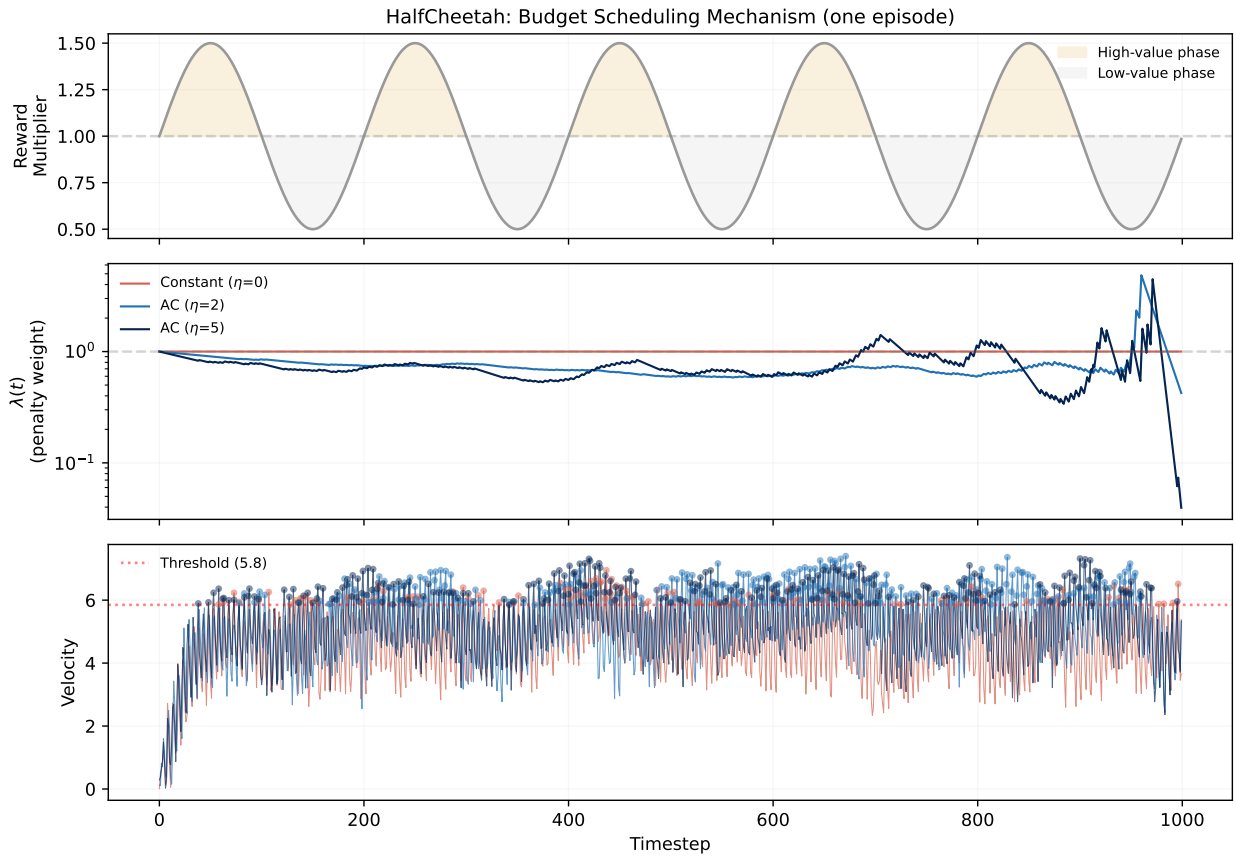

Figure 1: Budget scheduling mechanism over one HalfCheetah episode. Top: time-varying reward multiplier (gold = high-value phases). Middle: penalty weight $\lambda(t)$ — constant for $\eta = 0$ (red), adaptive for $\eta > 0$ (blue). λ decreases when budget is plentiful and spikes near depletion ($\eta = 5$, late episode). Bottom: velocity traces with threshold violations (dots).

6.1 Budget Satisfaction and Reward Retention

Table 1 summarizes the main results, averaged over 5 training seeds (HalfCheetah) and 3 training seeds (wind farm) with 10 evaluation episodes per seed. In HalfCheetah, the AC schedule ($\eta = 2$) retains $88.9\% \pm 2.3\%$ of unconstrained reward while using 97% of the available velocity budget, compared to only $84.3\% \pm 3.4\%$ reward and 63% budget utilization for the constant penalty ($\eta = 0$). In the wind farm domain, the unconstrained agent uses negative yaw on 116 ± 38 out of 200 steps for the upstream turbine—the optimal policy relies heavily on wake steering. With the AC schedule, negative yaw drops to within the budget while maintaining power close to the unconstrained level.

The key comparison in Figure 3 shows the structural advantage of adaptive scheduling. Constant-penalty configurations face a dilemma: low guidance scale under-constrains (budget exceeded) while high guidance scale over-constrains (budget wasted, reward lost). The AC schedule resolves this by adapting the penalty to the budget state, simultaneously achieving high utilization and high reward.

6.2 Hard Guard Ablation

A central question: does the AC schedule itself prevent budget violations, or is the hard guard ($C^i < B^i$) necessary? Table 2 shows the answer on HalfCheetah: the AC schedule alone provides virtually identical budget management to AC + hard guard (244/250 violations in both cases). The guard-only baseline ($\eta = 0$,

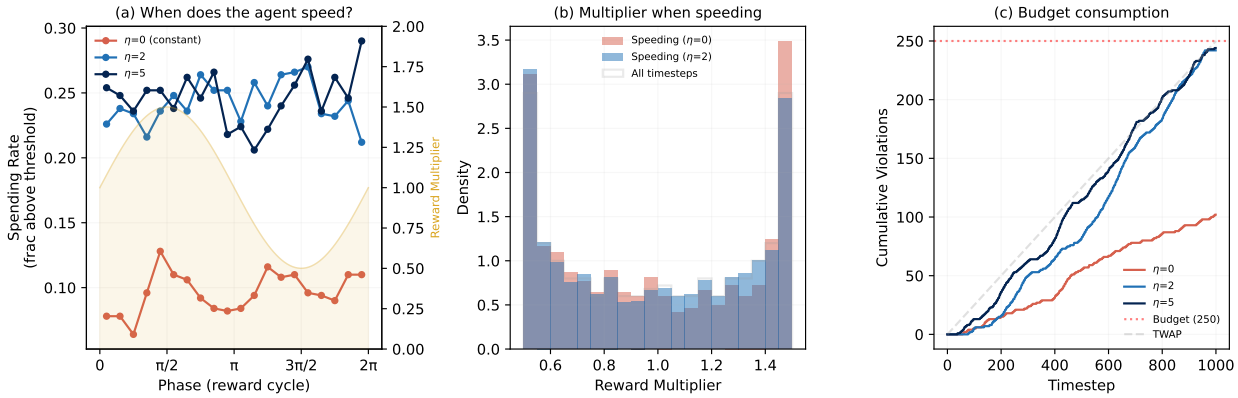

Figure 2: When does the agent speed? (a) Spending rate by reward cycle phase: the AC schedule ($\eta \geq 2$) sustains a uniformly high spending rate ($\sim$25%), while the constant penalty ($\eta = 0$) spends only $\sim$10%. (b) Reward multiplier distribution at speeding timesteps. (c) Cumulative budget consumption: $\eta = 0$ uses only half the budget; AC uses 96–99%.

Table 1: Main results across both domains (mean $\pm$ std over training seeds). Budget utilization reports the fraction of allowed budget actually spent. Higher utilization with high reward indicates intelligent allocation.

HalfCheetah (5 seeds)	Reward	% Unconstrained	Budget Used
Unconstrained	4229 ± 468	100%	—
Constant ($\eta = 0$)	3556 ± 358	84.3 ± 3.4%	63%
AC ($\eta = 2$)	3759 ± 440	88.9 ± 2.3%	97%
AC only (no guard)	3752 ± 434	88.7 ± 2.6%	96%
Wind Farm (3 seeds)	Power (MW)	NegYaw (T0)	NegYaw (T1)
Unconstrained	24.7 ± 1.2	116 ± 38	109 ± 35
Hard clip (no neg yaw)	25.7 ± 0.7	0	0
AC ($\eta = 2$, $k = 2$, gs=0.05)	26.5 ± 1.4	10 ± 2	10 ± 2
Constant ($\eta = 0$, gs=0.1)	24.3 ± 2.1	76 ± 25	68 ± 30

constant penalty) uses only 54% of the budget—it constrains but does not allocate.

6.3 Schedule Comparison: $w^*(u) = 1/u$ vs. $w(u) = \exp(\eta(1/u - 1))$

Figure 5(b) compares the theoretically optimal $u^{-\eta}$ schedule against our practical $\exp(\eta(1/u - 1))$ approximation. Both achieve similar reward retention across all η values. The optimal schedule is marginally better at $\eta = 2$ (87.0% vs. 86.6%), confirming that the practical form is a faithful approximation. The difference is smaller than the inter-episode variance, supporting our use of the more numerically stable exponential form.

6.4 Budget Flexibility

Figure 4 demonstrates the "train once, constrain at deployment" property. A single HalfCheetah policy, trained unconstrained, is evaluated at five budget levels from 10% to 75% of episode length. The reward curve is concave and bows above the linear baseline: at 25% budget, the agent retains 86.5% of unconstrained reward (not 25%). This disproportionate value capture confirms that the AC schedule concentrates spending into high-value timesteps.

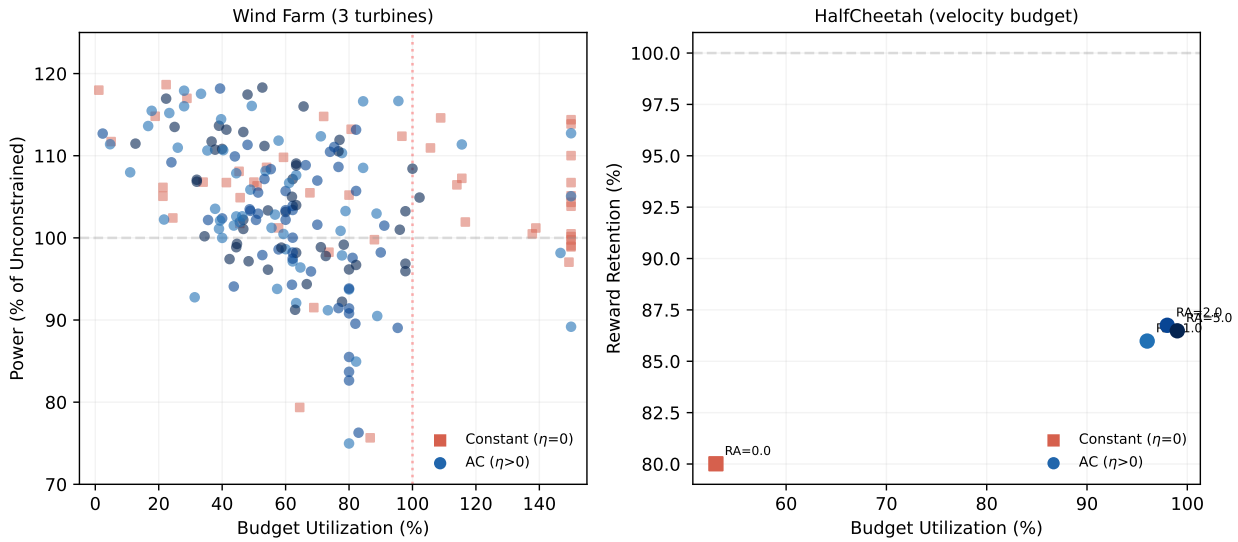

Figure 3: Budget utilization vs. reward retention. Constant penalty (squares, red) cannot simultaneously achieve high utilization and high reward. AC schedule (circles, blue shades by η) clusters in the efficient top-right region.

Table 2: Hard guard ablation (HalfCheetah, budget=250/1000 steps, 5 seeds). The AC schedule alone (no hard guard) provides the same budget management as the full system.

Configuration	% Unconstrained	Budget Used	Reward
Both (AC + guard)	$88.9 \pm 2.3\%$	$96.8 \pm 1.9\%$	3759 ± 440
AC only (no guard)	$88.7 \pm 2.6\%$	$96.4 \pm 2.0\%$	3752 ± 434
Guard only ($\eta = 0$)	$84.3 \pm 3.4\%$	$63.0 \pm 18.8\%$	3556 ± 358

6.5 Comparison to Saute RL (Retrained Baseline)

Table 3 compares our post-hoc method to Saute RL [Sootla et al., 2022], which augments the state with remaining budget and retrains the policy. At the 25% budget level where Saute was trained, it achieves 2% higher reward than our method—but *violates the budget* (305 violations vs. 250 allowed). Our AC schedule ($\eta = 5$) stays within budget (246/250) while retaining 90% of unconstrained reward.

The decisive advantage is flexibility: our method handles budget=10% and budget=50% with the same policy and zero additional training. Saute RL would require retraining a separate policy for each budget level.

7 Related Work

Constrained MDPs. The Lagrangian approach to CMDPs [Altman, 1999] learns a dual variable μ via primal-dual optimization. PID-based methods [Stooke et al., 2020] improve stability. Saute RL [Sootla et al., 2022] augments the state with remaining budget but requires retraining. Predictive Lagrangian Optimization [Chen et al., 2025] forecasts future multipliers but integrates with training. CAPS [Yao et al., 2025] switches between pre-trained policies for different constraint levels. Budget-Conditioned Reachability [Liu et al., 2026] conditions on remaining budget via reachability analysis for offline safe RL. All require training or pre-training for each constraint specification; our method requires none.

Post-hoc constraint composition. Composable Energy Policies [Urain et al., 2023] compose energy functions for robotic manipulation. Constrained Diffusers [Zhang et al., 2025] add safety constraints via

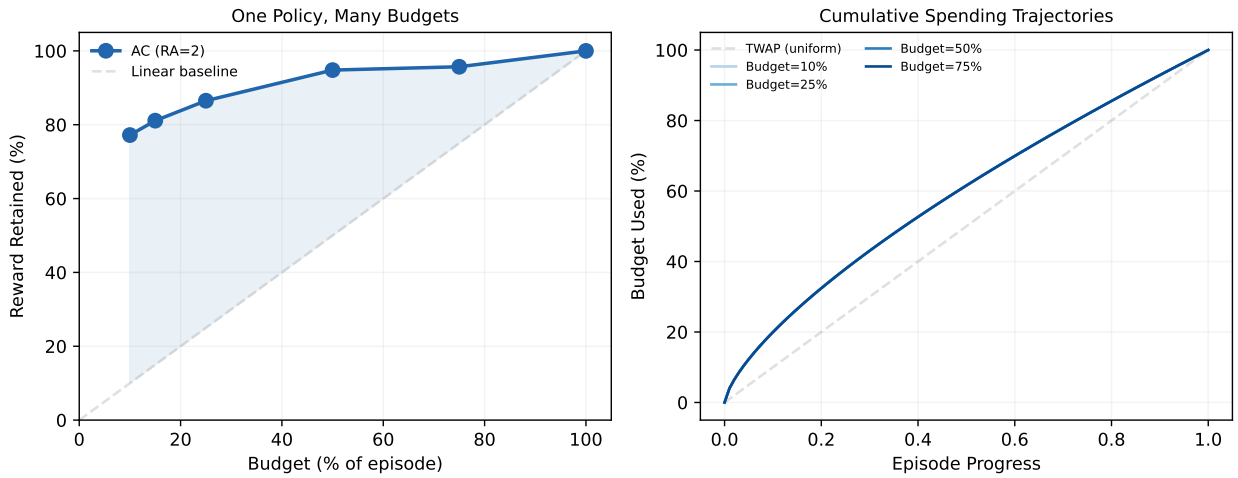

Figure 4: Budget flexibility. Left: same policy at different budget levels (concave curve = intelligent allocation). Right: cumulative spending trajectories depart from the TWAP diagonal, spending conservatively early and catching up during high-value phases.

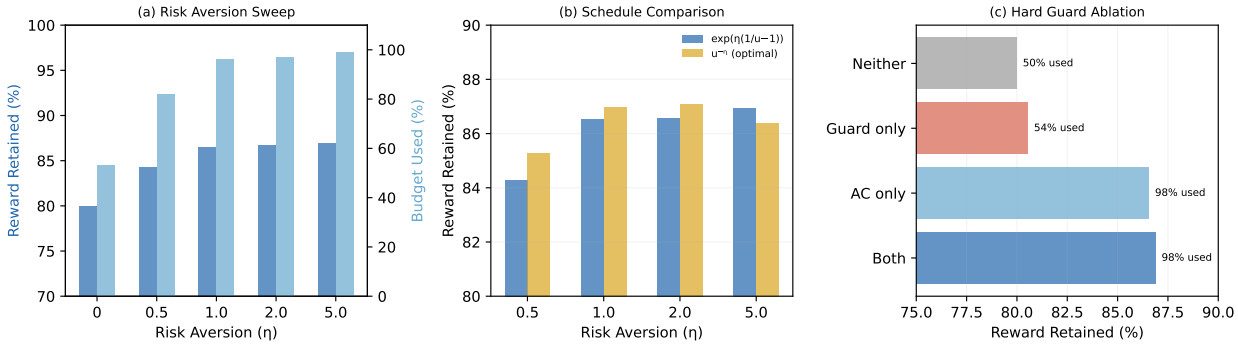

Figure 5: Ablations on HalfCheetah. (a) Risk aversion sweep: $\eta = 0$ wastes budget (53% used), $\eta \geq 1$ achieves >96% utilization and 7% more reward. (b) Schedule comparison: $u^{-\eta}$ (optimal) and $\exp(\eta(1/u - 1))$ (practical) perform comparably. (c) Hard guard ablation: the AC schedule alone matches the full system.

classifier guidance. SafeDiffuser [Xiao et al., 2025] uses control barrier functions. FISOR [Zheng et al., 2024] learns a feasibility classifier for diffusion-based safe RL. PICNN [Durkin et al., 2025] corrects actions via input-convex networks. All handle *instantaneous* constraints; none address cumulative budgets that span an episode.

Budget pacing. Online budget allocation is well-studied in advertising [Balseiro et al., 2024], where PID controllers and multiplicative updates pace daily spend. Bandits with knapsacks [Agrawal and Devanur, 2014] provide regret bounds for combinatorial budget problems. Our work transfers this structure to RL deployment via energy composition.

Optimal execution. Almgren and Chriss [Almgren and Chriss, 2001] derive optimal liquidation trajectories from mean-variance objectives. Recent work combines RL with execution [Ning et al., 2024]. We draw on the urgency-based scheduling intuition but derive our formula from Boltzmann budget balance rather than market impact models.

Table 3: Saute RL (retrained) vs. post-hoc AC scheduling on HalfCheetah. Saute achieves slightly higher reward but violates the budget. AC satisfies the budget and generalizes to unseen budget levels without retraining.

Method	Budget	Reward	Violations	Retrained?
Saute RL	25%	4711	305/250 ×	Yes
AC ($\eta = 5$)	25%	4630	246/250 ✓	No
AC ($\eta = 2$)	25%	4607	244/250 ✓	No
AC ($\eta = 2$)	10%	4151	98/100 ✓	No
AC ($\eta = 5$)	50%	5162	442/500 ✓	No

8 Conclusion

Cumulative budget constraints are ubiquitous in deployed RL but underserved by existing methods. We showed they can be enforced post-hoc via a theoretically grounded, closed-form penalty schedule that composes with pre-trained energy-based policies. The method requires zero retraining, supports heterogeneous per-agent budgets, and adapts instantly to changing specifications. Across wind farm yaw control and MuJoCo locomotion, the adaptive schedule uses budget more effectively than constant penalties—concentrating risky actions into high-value conditions—while maintaining constraint satisfaction.

Limitations. The Boltzmann response assumption (Proposition 1) is approximate; the true policy response depends on the energy landscape geometry. The method works best when the base penalty scale is calibrated relative to the learned energy head. The theoretical guarantee is for the fluid approximation; finite-sample constraint satisfaction relies on the exponential wall backstop.

Future work. Extending to continuous cost functions (not just binary indicators), learned risk-aversion scheduling, and multi-agent coordination where agents share a collective budget.

References

Shipra Agrawal and Nikhil R Devanur. Bandits with concave rewards and convex knapsacks. In *ACM Conference on Economics and Computation*, 2014.

Robert Almgren and Neil Chriss. Optimal execution of portfolio transactions. *Journal of Risk*, 3:5–39, 2001.

Eitan Altman. *Constrained Markov decision processes*. Chapman and Hall/CRC, 1999.

Santiago Balseiro et al. A field guide for pacing budget and ROS constraints. *International Conference on Machine Learning*, 2024.

Yuxiang Chen et al. Predictive Lagrangian optimization for constrained reinforcement learning. *arXiv preprint arXiv:2501.15217*, 2025.

Shashank Durkin et al. Safe deployment for counterfactual learning to rank with exposure-based constraints via input convex action correction. *arXiv preprint arXiv:2507.22640*, 2025.

Ruairidh Gladstone et al. Energy-based transformers are scalable learners and thinkers. *arXiv preprint arXiv:2507.02092*, 2025.

Chia-Chi Liu et al. Beyond hard constraints: Budget-conditioned reachability for safe offline reinforcement learning. *arXiv preprint arXiv:2603.22292*, 2026.

Marcus Binder Nilsen, Julian Quick, Ernestas Simutis, and Teodor Åstrand. WindGym: A wind farm reinforcement learning environment. `https://github.com/DTUWindEnergy/WindGym`, 2025.

Boming Ning et al. Reinforcement learning for optimal execution: A survey. *arXiv preprint arXiv:2411.06389*, 2024.

Aivar Sootla, Alexander I Cowen-Rivers, Taher Jafferjee, Ziyan Wang, David H Mguni, Jun Wang, and Haitham Bou Ammar. Sauté RL: Almost surely safe reinforcement learning using state augmentation. In *International Conference on Machine Learning*, 2022.

Adam Stooke, Joshua Achiam, and Pieter Abbeel. Responsive safety in reinforcement learning by PID Lagrangian methods. In *International Conference on Machine Learning*, 2020.

Emanuel Todorov, Tom Erez, and Yuval Tassa. MuJoCo: A physics engine for model-based control. In *IEEE/RSJ International Conference on Intelligent Robots and Systems*, 2012.

Julen Urain, Anqi Li, Puze Liu, Carlo D'Eramo, and Jan Peters. Composable energy policies for reactive motion generation and reinforcement learning. In *International Journal of Robotics Research*, 2023.

Mike van Dijk et al. Load assessment of a wind farm considering negative and positive yaw misalignment for wake steering. *Wind Energy Science*, 10:1033–1051, 2025.

Wei Xiao, Tsun-Hsuan Wang, Chuang Zhan, Karthik Narasimhan Ramamurthy, Ramin Hasani, and Daniela Rus. SafeDiffuser: Safe planning with diffusion probabilistic models. *International Conference on Learning Representations*, 2025.

Zuxin Yao et al. Constraint-adaptive policy switching for offline safe reinforcement learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 2025.

Zheyu Zhang et al. Constrained diffusers for safe planning and control. *Advances in Neural Information Processing Systems*, 2025.

Zuxin Zheng et al. FISOR: Building safe and adaptable offline reinforcement learning agents with feasibility-guided diffusion model. *International Conference on Learning Representations*, 2024.